Reflected POST XSS in mlflow/mlflow

Valid Reported on Oct 16th 2023

Description

The vulnerability allows an attacker to inject malicious code into the Content-Type header of a POST request, which is then reflected back to the user without proper sanitization or escaping.

Proof of Concept

Here's the reflection:

and here is a script to reproduce the POC:

```
from requests import post

url = 'http://127.0.0.1:5000/api/2.0/mlflow/users/create'

headers = {
    'Content-Type': '<script>alert(document.domain)</script>',
}

resp = post(url, headers=headers).text
print(resp)
```

Impact

This can be exploited by an attacker to execute arbitrary JavaScript code in the context of the victim's browser, potentially compromising user sessions, stealing sensitive information, or performing other malicious actions on the user's behalf.

Occurrences

__init__.py L765

The `Content-Type` header of the user is directly injected in python formatted string and returned

```
make_response(f"Invalid content type: '{content_type}'", 400)
```

We are processing your report and will contact the **mlflow** team within 24 hours. 2 years ago

A **GitHub Issue** asking the maintainers to create a `SECURITY.md` exists 2 years ago

We have contacted a member of the **mlflow** team and are waiting to hear back 2 years ago

Mokrane Abdelmalek commented 2 years ago — Researcher

Hello, it has been a while.

Dan McInerney modified the Severity from Medium (6.5) to Medium (6.5) 2 years ago

Dan McInerney commented 2 years ago — Admin

Hi Mokrane,

While this looks valid, we only manually validate vulnerabilities above 7.0 CVSS score. I have adjusted the CVSS score for accuracy on a POST XSS vulnerability and it comes to 6.5. Due to this, we will leave this report open until MLflow maintainers respond that they'd like a CVE issued, or you may mark informational and allow it to go public.

Thanks, Dan McInerney

Ben Wilson commented 2 years ago — Maintainer

Wouldn't this mechanism require that an admin of an MLflow deployment execute this attack? I'm afraid I don't quite follow how this is a vulnerability. Could you provide further details on what the exact mechanism would be for this in regards to the creation of a user account?

Mokrane Abdelmalek commented 2 years ago — Researcher

Thank you for your response. While the exploitation of this issue does not necessitate admin privileges, it does rely on tricking any user into sending a request with a manipulated `Content-Type` header. The crux of the matter lies in the absence of proper sanitization of user input, which allows an attacker to inject and reflect malicious JavaScript code.

I acknowledge the difficulty in crafting a specific exploitation scenario in this case. The authentication mechanism, relying on HTTP authentication with cached credentials in the browser, adds a layer of complexity. Additionally, the absence of the `Authorization` header in CORS-prevented scenarios prevents successful exploitation, as the backend responds with a 400 status for preflight check requests lacking the necessary authorization header.

It's worth noting that while the current setup may mitigate the immediate risk, the lack of CSRF protection, combined with the absence of input sanitization, still poses a potential threat. The importance of defense in depth cannot be understated, and any vulnerability that reduces the depth of defense should be considered a valid finding. As security is an evolving landscape, addressing these concerns proactively can contribute to a more robust and resilient system in the long run.

If there are any additional details or specific aspects you'd like further clarification on, please let me know.

Yuki Watanabe commented 2 years ago — Maintainer

Thank you for the detailed response, Mokrane. I understand that we need a better authorization mechanism in long term, but as a a quick mitigation for this vulnerability, I'm adding a simple character-based sanitization before printing back content-type value: https://github.com/mlflow/mlflow/pull/10526. As I'm not an expert on this domain, it would be great if you could take a look and see if that works as a guardrail for this vulnerability. Thank you.

Mokrane Abdelmalek commented 2 years ago — Researcher

Thank you for your prompt response and the quick mitigation effort. I've reviewed the proposed fix in your pull request, and I don't see a straightforward way to bypass the implemented regex. The restriction on `<>` characters effectively mitigates the injection of JavaScript code.

While the regex-based approach seems effective, considering the principle of least privilege, I would like to suggest an alternative approach. Instead of reflecting the manipulated Content-Type value and then sanitizing it, we could simply reject invalid `Content-Type` values outright, providing an error message without echoing the user input. This eliminates the need for sanitization and potential risks associated with reflecting user input.

Alternatively, if there is a specific reason for reflecting the `Content-Type` value, it might be worthwhile to leverage well-established libraries for HTML escaping, such as `jinja.escape`, to ensure a robust and widely accepted sanitization process.

Regarding the `Authorization` header authentication, I appreciate the clarification. My earlier mention was to highlight the current difficulty in bypassing the required authorization header, not to suggest a specific vulnerability.

Yuki Watanabe commented 2 years ago — Maintainer

Thank you so much for the review and suggestion, Makrane. Not including the user input in the error message makes sense to me, there should not be a significant reason to echo it. I updated the PR and will resolve this vulnerability once it's released.

Mokrane Abdelmalek commented 2 years ago — Researcher

You're welcome! I'm glad the suggestion makes sense to you. Regarding the CVE, is it going to be requested or could you please share how you plan to proceed in that regard? Thank you.

Dan McInerney gave praise 2 years ago

Thank you for the finding!

The researcher's credibility has slightly increased as a result of the maintainer's thanks: +1

Mokrane Abdelmalek commented 2 years ago — Researcher

Hello, I wanted to follow up on my previous inquiry about the CVE and how you plan to proceed in that regard. Any updates or information you could provide would be greatly appreciated. Thank you.

The researcher has received a minor penalty to their credibility for miscalculating the severity: -1

Harutaka Kawamura validated this vulnerability 2 years ago

m0kr4n3 has been awarded the disclosure bounty ✓

The fix bounty is now up for grabs

The researcher's credibility has increased: +7

Ben Wilson commented 2 years ago — Maintainer

Hi Mokrane, we've released the fix for this in 2.9.0 and we're preparing the CVE submission tonight. Thanks again for the studious work!

Ben Wilson gave praise 2 years ago

Thank you for the finding!

The researcher's credibility has slightly increased as a result of the maintainer's thanks: +1

Yuki Watanabe marked this as fixed in **2.9.0** with commit **28ff3f** 2 years ago

$ **Yuki Watanabe** has been awarded the fix bounty ✓

This vulnerability has now been published 2 years ago

$ **__init__.py#L765** has been validated ✓

Sign in to join this conversation

CVE
CVE-2023-6568
Published

Vulnerability Type
CWE-79: Cross-site Scripting (XSS) - Reflected

Severity
Medium (6.5)

Attack vector	Network
Attack complexity	Low
Privileges required	None
User interaction	Required
Scope	Unchanged
Confidentiality	High
Integrity	None
Availability	None

Open in visual CVSS calculator

Registry
Pypi

Affected Version
2.7.1

Visibility
Public

Status
Fixed

Disclosure Bounty
$125

Fix Bounty
$31.25

Found by

Mokrane Abdelmalek
@m0kr4n3
MIDDLEWEIGHT

Fixed by

Yuki Watanabe

@b-step62

UNPROVEN

Supported by Palo Alto Networks and Prisma AIRS, leading the way to greater AI security.